

Generación autónoma de microservicios

Dónde está la industria, cómo funciona la solución, qué cuesta y hacia dónde vamos

Ingeniería AgentIA

2026

Dónde está la industria: tres caminos

1. Agentes generalistas de código

Arreglan o completan código *ya existente*.

- Dominan los rankings.
- Saturados y **caros** (\$5–\$30 por tarea).

2. Generadores guiados por especificación

Construyen la aplicación *desde una especificación formal*.

- Es nuestro camino.
- Solo se resuelve **12–55%**: mucho espacio.

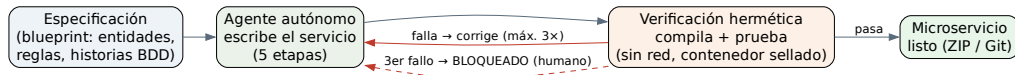
3. Agentes que se auto-mejoran

Escriben su propio “manual” de trabajo.

- Prometedor en laboratorio.
- En código real, mejora **casi nula**.

La lección de toda la investigación: el ingrediente que gana no es un modelo más listo ni un manual auto-escrito — es una *verificación* de fiar.

Cómo funciona la solución



Lo que eso garantiza

- **Se escribe** un servicio completo, no fragmentos: entidades, lógica, API, tests.
- **Se verifica** en un contenedor sellado, sin red: compila y pasa sus pruebas, o no se entrega.
- **Se autocorrigie** hasta 3 veces; al tercer fallo se detiene y pide a un humano. Nunca un bucle infinito, nunca un “éxito” no comprobado.
- **Se audita** el cumplimiento de las reglas del proyecto, aplicadas por la plataforma, no por la opinión del modelo.

En una frase

Una especificación entra; un microservicio compilado, probado y conforme sale — o el sistema dice que no pudo.

El coste (medido, de la última ejecución)

Qué	Coste
Un microservicio (dificultad media, recién ejecutado)	\$0.067
Rango por servicio (corpus de 10 servicios)	\$0.052 – \$0.077
Un servicio garantizado (se generan 3, se entrega el que compila)	\$0.20

- ~\$0.005 por llamada; ~**78K tokens por servicio** (15.6K por llamada), salida $\approx 1.9\times$ la entrada.
- Uso acumulado: **19.0M tokens** (6.5M entrada + 12.5M salida) en **1,219 llamadas**.
- El coste escala con el tamaño del servicio, de forma predecible.
- Fuente: sistema de registro propio y su espejo MLflow (ya operativo).

- **La plataforma ya verifica su propia salida.** Antes reportaba “0 de 15 verificadas”; el bloqueo era un defecto de un carácter en la configuración de despliegue.
- **Ambas vías de producto compilan y prueban,** no solo una.
- **La métrica de calidad dejó de ser manipulable:** ya no se mejora “generando más archivos”.
- **El registro de coste es honesto** y ahora visible en MLflow.
- **Queda un punto abierto:** el verificador a veces cambia su veredicto (se corrige en la siguiente iteración).

El futuro: auto-mejora

La idea

Que el sistema aprenda de sus propios errores y mejore su “manual” de generación automáticamente.

Lo que la evidencia exige

Un sistema que aprende de un juez ruidoso aprende ruido. Por eso el orden correcto es:

- ❶ Hacer **sólido el verificador** (que dé siempre el mismo veredicto).
- ❷ Hacer que la **medida detecte** los fallos reales.
- ❸ *Solo entonces*, encender el bucle de auto-mejora.

El mecanismo de auto-mejora **ya está construido**; se enciende cuando la base sea fiable.

Paso	Qué	Esfuerzo
1	Sólido el verificador (una configuración)	corto
2	Detectar fallos reales en la medida	corto
3	Medir si un “manual” aporta	\$3–\$5
4	Encender la auto-mejora (ya construida)	condicionado

El mensaje

De “no podía verificar su propia salida” a “genera, compila y verifica por \$0.07, y se auto-mejorará cuando la base sea fiable”.